Chapter 3 Analysis Phase

3.1 Requirements

This section defines what the Calma Mental Health Support Platform must do (functional requirements) and the quality attributes it must satisfy (non-functional requirements). The platform serves three user roles: Patient, Doctor, and Admin.

3.1.1 Functional Requirements - what system should do:

- The system should support three user roles (Patient, Doctor, Admin) with strict authorization, issuing secure access tokens through Laravel Sanctum at login.
- The system should allow all users to register, log in, edit their profile information, and delete their account.
- The system should route each user to the interface that matches their role after login, and reject any request that violates role restrictions (for example, an Admin attempting to log in through the mobile API).
- The system should provide an AI Chatbot that generates empathetic responses to patients in Egyptian Arabic, Modern Standard Arabic, and English, through both text messages and voice messages.
- The system should allow patients to record a voice message inside the chatbot, convert the speech to text using a Speech-to-Text (STT) service hosted on the Python microservice, and process the resulting transcript through the same NLP pipeline as typed messages.
- The system should analyze every chat message (typed or transcribed from voice) using an NLP model (BERT) hosted on a Python microservice, classify its intent, and return a danger flag (is_danger) to the client.
- The system should instantly display an un-exitable red “SOS Screen” showing emergency support numbers and prevent backward navigation when self-harm intent is detected in a typed or spoken message.
- The system should store high-risk texts (including transcripts of voice messages) anonymously in a data warehouse to improve the accuracy of the AI model in the future.
- The system should store voice recordings in a secure location isolated from public directories, and make them accessible only to the patient who recorded them.
- The system should automatically collect mental health articles and books from trusted global sources (web scraping or RSS feeds) using scheduled jobs, and store their title, summary, link, and image.

- The system should categorize collected articles (for example: anxiety, sleep, depression) so patients can find them easily.
- The system should allow approved doctors to publish awareness posts and practical tips, and allow patients to read them.
- The system should allow patients to publish personal-experience posts anonymously: the author's name must never be displayed or returned to other users, while the author's identity is kept internally for moderation purposes only.
- The system should allow the Admin to review, hide, or delete any community post that violates the platform's safety rules.
- The system should store uploaded doctor documents in a secure location isolated from public directories and link them to the database.
- The system should send real-time notifications (Firebase Cloud Messaging or Pusher) when a doctor’s account status changes.

3.1.2 Patient functions:

- The patient should be able to use the system through the mobile application (Flutter) or the web platform (React).
- The patient should be able to chat with the AI Chatbot at any time by typing a message and receive supportive replies.
- The patient should be able to record a voice message in the chatbot, see the transcribed text of the message, and receive an empathetic reply (or the red SOS screen if danger is detected).
- The patient should be able to press the SOS emergency button, which is always available, to open the red emergency screen immediately without needing a conversation.
- The patient should be able to record their daily mood on a scale from 1 (sad) to 5 (happy).
- The patient should receive a pop-up with a random positive challenge (such as breathing exercises, walking, or reading) when a low mood score (1 or 2) is recorded.
- The patient should be able to view weekly and monthly mood reports with interactive bar and line charts, including the average and a comparison with the previous month.
- The patient should be able to listen to a static library of calming sounds for relaxation.
- The patient should be able to read the categorized mental health articles and books.

- The patient should be able to read the awareness posts and tips published by approved doctors, and learn from them.
- The patient should be able to publish a post about a personal experience to help others, with the post appearing anonymously (without the patient's name) to guarantee complete privacy.
- The patient should be able to browse the list of approved doctors, view their specialization and experience, and contact a doctor through a phone number or an external clinic link.

3.1.3 Doctor functions:

- The doctor should be able to register through a multi-step form providing name, age, date of birth, skills, specialization, personal photo, national ID (front and back), and graduation certificate.
- The doctor should be able to upload sensitive documents securely.
- The doctor's account should remain in a "pending" state after submission, and the doctor should see a clear waiting screen (24-hour review) instead of the dashboard.
- The doctor should be notified and shown a welcome screen once the Admin approves the account.
- The doctor should be able to view the submitted data in read-only mode while the account is pending.
- The approved doctor should be able to update their profile and contact links (phone numbers, clinic URL) that appear to patients.
- The approved doctor should be able to publish awareness posts and practical tips for patients, and to edit or delete their own posts. Doctors whose accounts are still pending cannot publish.

3.1.4 Admin functions:

- The admin should be able to log in through the web platform (React) only.
- The admin should be able to view system statistics such as the total number of patients, approved doctors, and new pending requests.
- The admin should be able to view the list of pending doctor requests and open each doctor's file, including the ID card and graduation certificate.
- The admin should be able to approve a doctor, which changes the doctor's status to active and makes the doctor appear immediately in the patients' application.
- The admin should be able to reject or delete a doctor whose data is incorrect.
- The admin should be able to manage (view and delete) doctor and patient accounts.

- The admin should be able to review community posts (doctor posts and anonymous patient posts) and hide or delete any inappropriate post.

3.1.5 Non-Functional Requirements

The quality attributes of the system and how the system should do what it does:

- **Performance:** The system should remain responsive, and the chatbot with its danger detection should return a response within a few seconds under normal load, including the time needed to transcribe a voice message.
- **Security:** The system should hash all passwords, protect every API with token-based authentication, enforce role-based access through middleware (for example, only approved doctors can publish doctor posts), and store sensitive documents and voice recordings in a non-public location.
- **Privacy:** Patient conversations, voice messages, and mood data should be accessible only to the patient, any text stored for model improvement should be fully anonymized, and the identity of a patient who publishes a community post should never be exposed to other users.
- **Moderation:** Every community post should be reviewable and removable by the Admin to keep the community space safe and supportive.
- **Reliability:** The SOS screen should trigger consistently whenever danger is detected, whether the message was typed or spoken, and the SOS button should always be available, even when the AI service is slow.
- **Usability:** The interface should be calming and intuitive, using rounded shapes and soft blue tones, offering a simple one-tap voice recording control, and reserving the red color exclusively for emergencies, especially for users in a stressful emotional state.
- **Scalability:** The AI models (NLP and Speech-to-Text) should run as an independent Python microservice separate from the Laravel backend, so each part can scale as the number of users grows.
- **Compatibility:** The mobile application should run on Android and iOS devices, and the web dashboards should work on modern web browsers. Voice recording requires microphone permission on both mobile and web.
- **Localization:** The system should support Arabic (right-to-left) and English interfaces and understand Egyptian dialect in chat, in both typed and spoken messages.
- **Availability:** The platform should be available around the clock to provide continuous (24/7) support for patients.
- **Maintainability:** The code should be modular, following a clear separation between the mobile app, web dashboard, backend API, and AI service.

- **Testability:** The system should be tested using PHPUnit for backend feature tests, Jest for React tests, and Postman for API testing.
- **Documentation:** Comprehensive documentation should be created for developers and future maintainers, with a focus on the API, database schema, AI service integration, and system architecture.

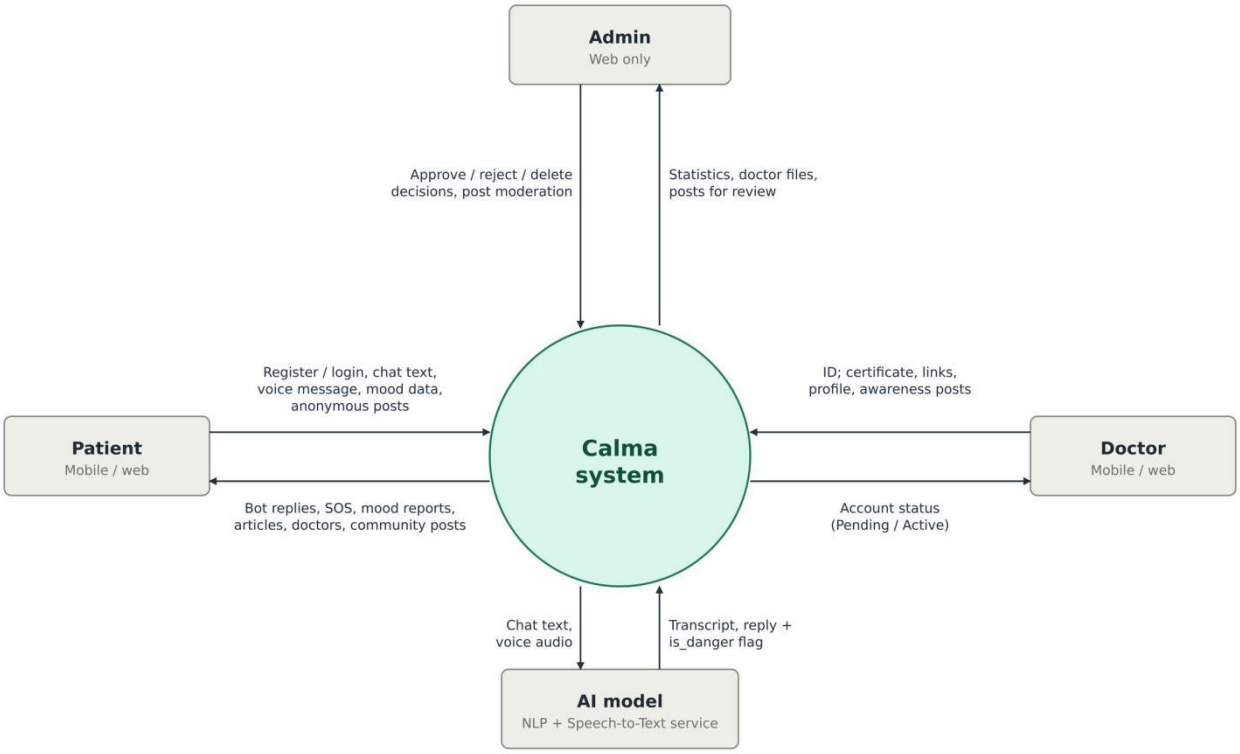
3.2 System Models

3.2.1 DFD (Data Flow Diagram)

A data flow diagram (DFD) shows how data moves between the people who use a system, the processes that work on that data, and the places where the data is stored. Calma has three kinds of users (Patient, Doctor, Admin) and one external AI service, so the diagrams below follow those four parties.

- **DFD Level 0 (Context Diagram)**

Level 0 treats the whole platform as one process and shows only what goes in and what comes out.

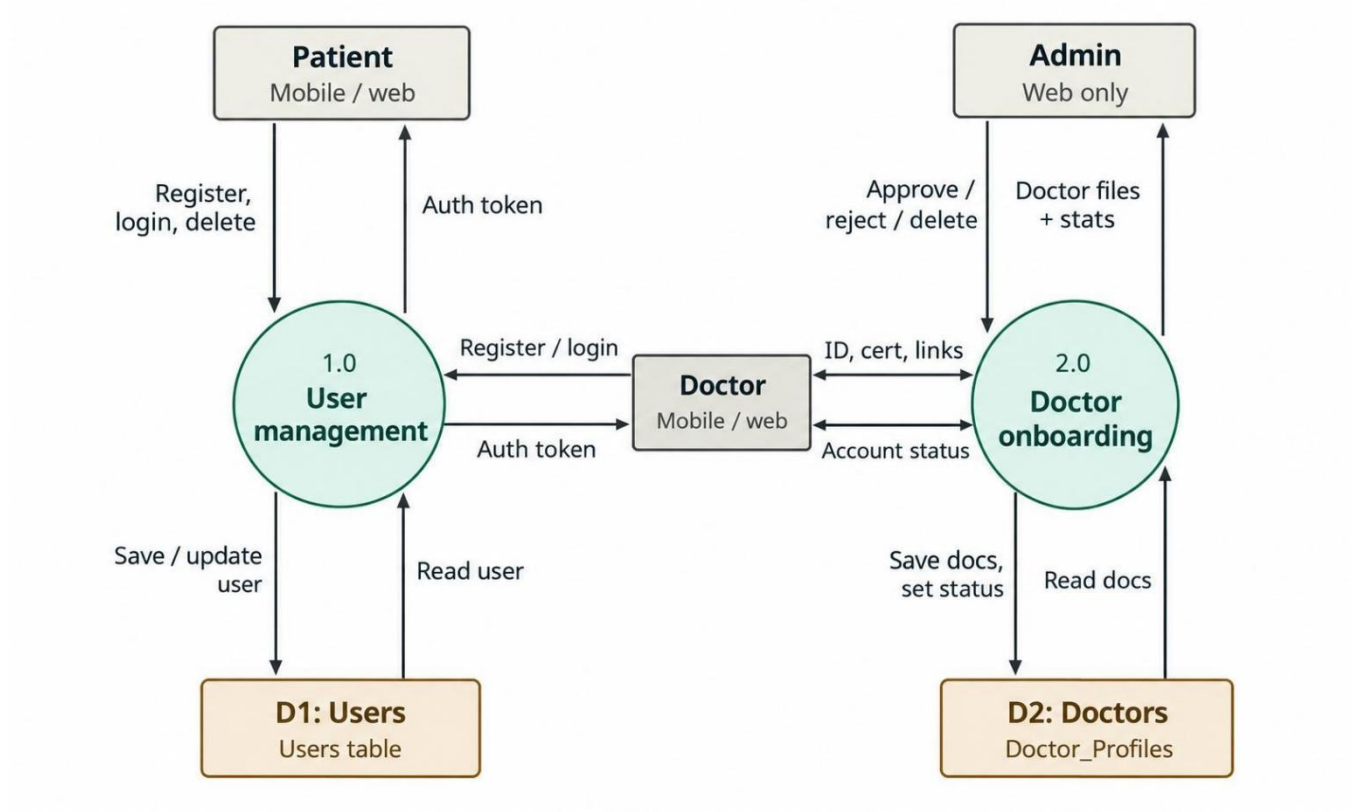


DFD Level 0 for the Calma system

- **DFD Level 1**

Part A : User management and doctor onboarding

Level 1 opens the single Level 0 process into six main processes. To keep the arrows readable, it is drawn in three parts. Part A covers the processes that deal with accounts and doctor verification.

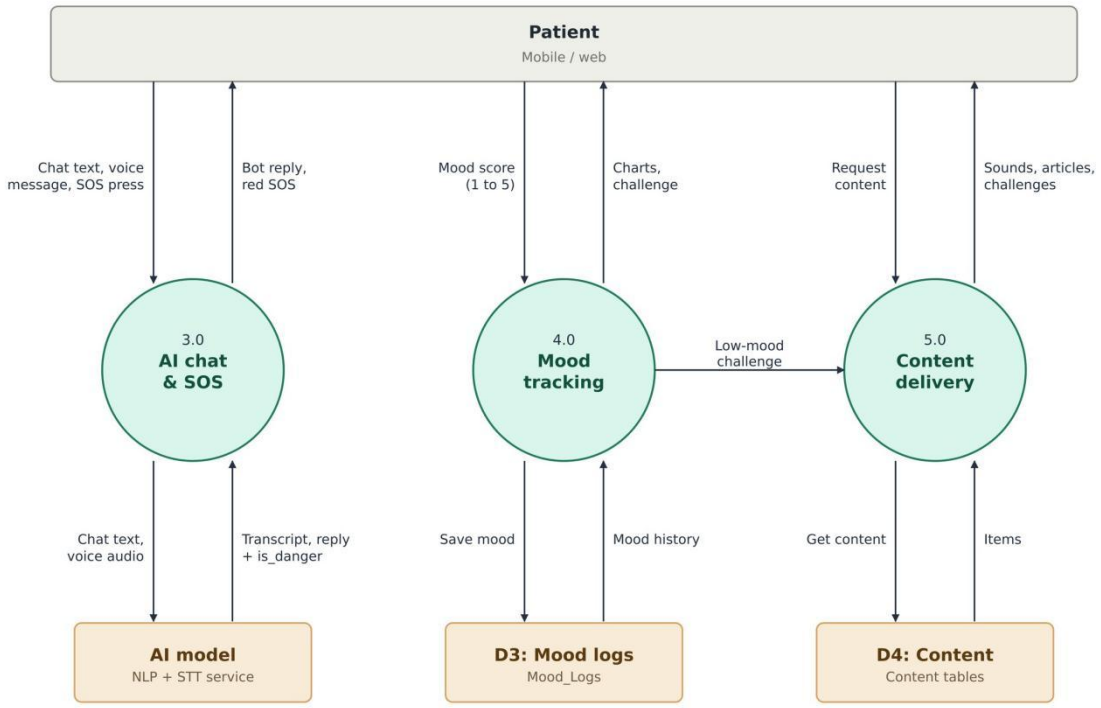


DFD Level 1 , Part A (processes 1.0 and 2.0)

- **DFD Level 1 (Continued)**

Part B : Patient Services

Part B covers what a patient does after logging in: chatting with the AI bot by text or voice, tracking mood, and reading the content library.

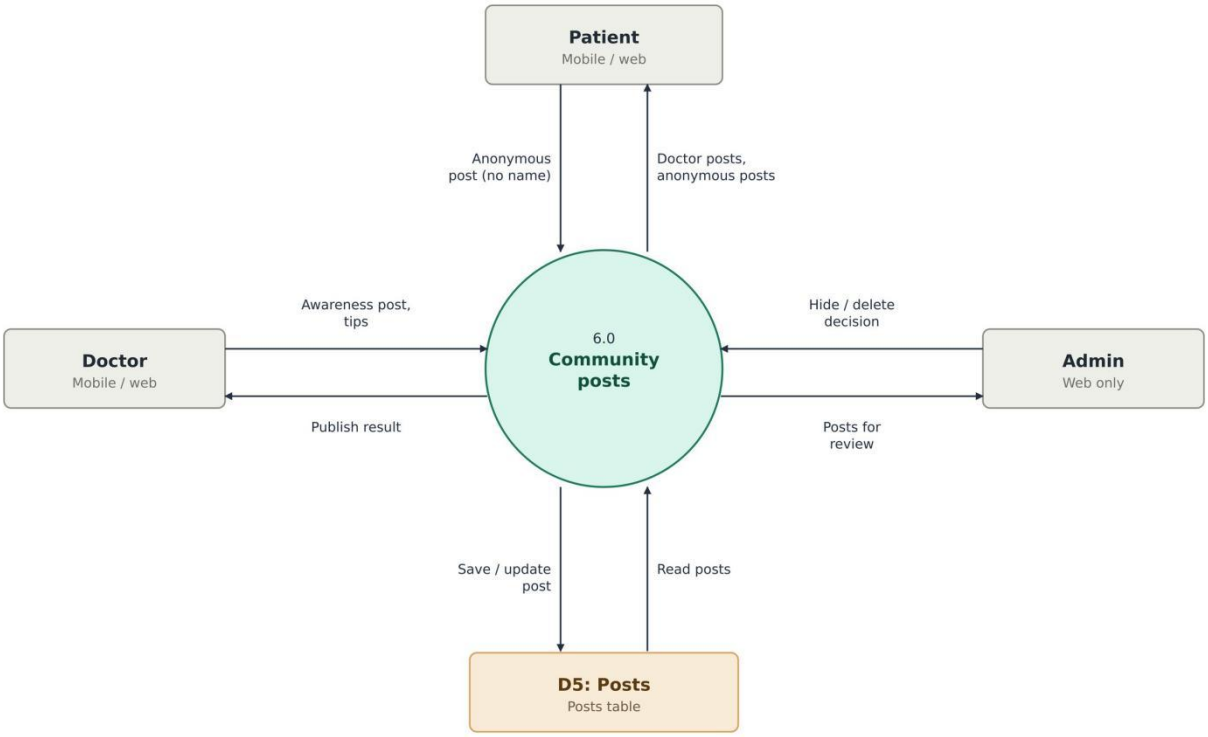


DFD Level 1, Part B (Processes 3.0, 4.0 and 5.0)

- **DFD Level 1 (Continued)**

Part C : Community Awareness Posts

Part C covers the community awareness space: approved doctors publish awareness posts, patients share their experiences anonymously and read the posts, and the Admin moderates all posts.

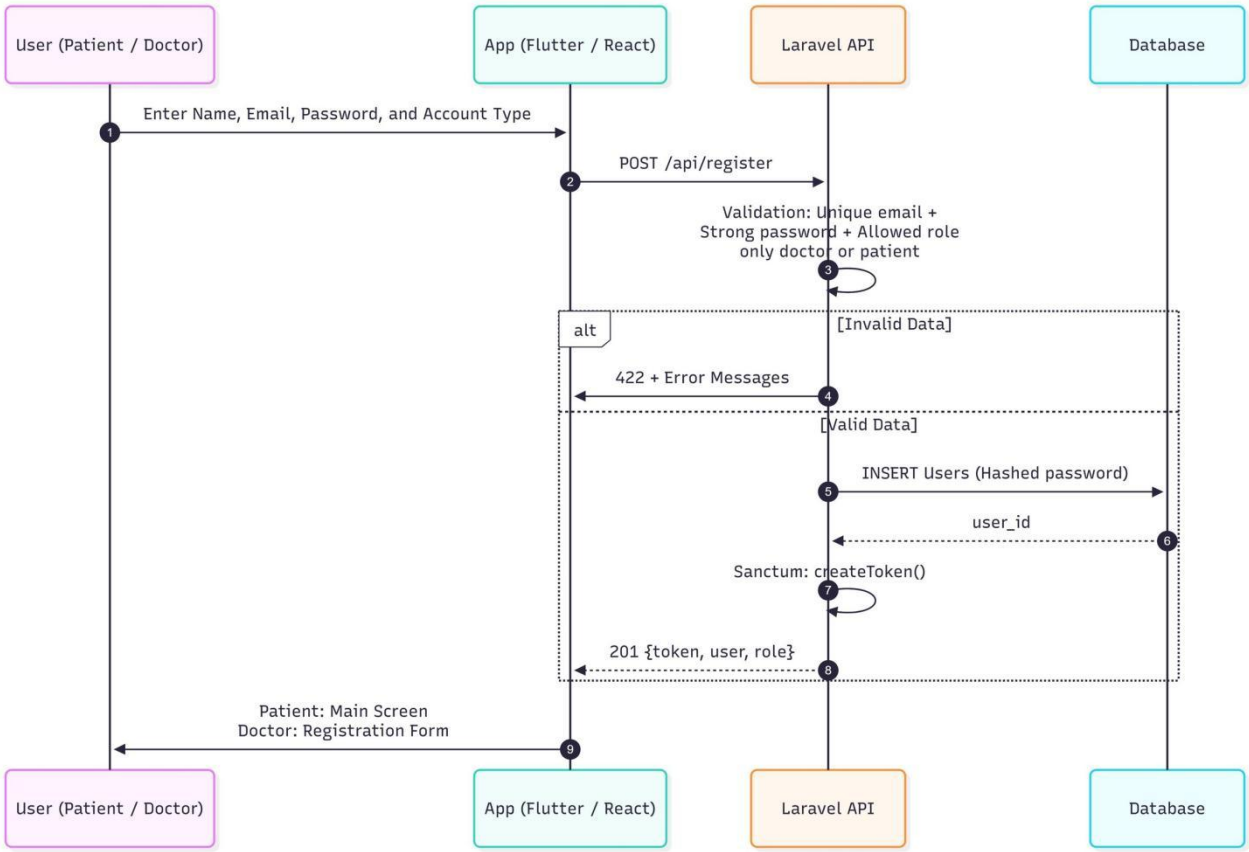


DFD Level 1, Part C (Process 6.0)

3.2.2 Sequence Diagram

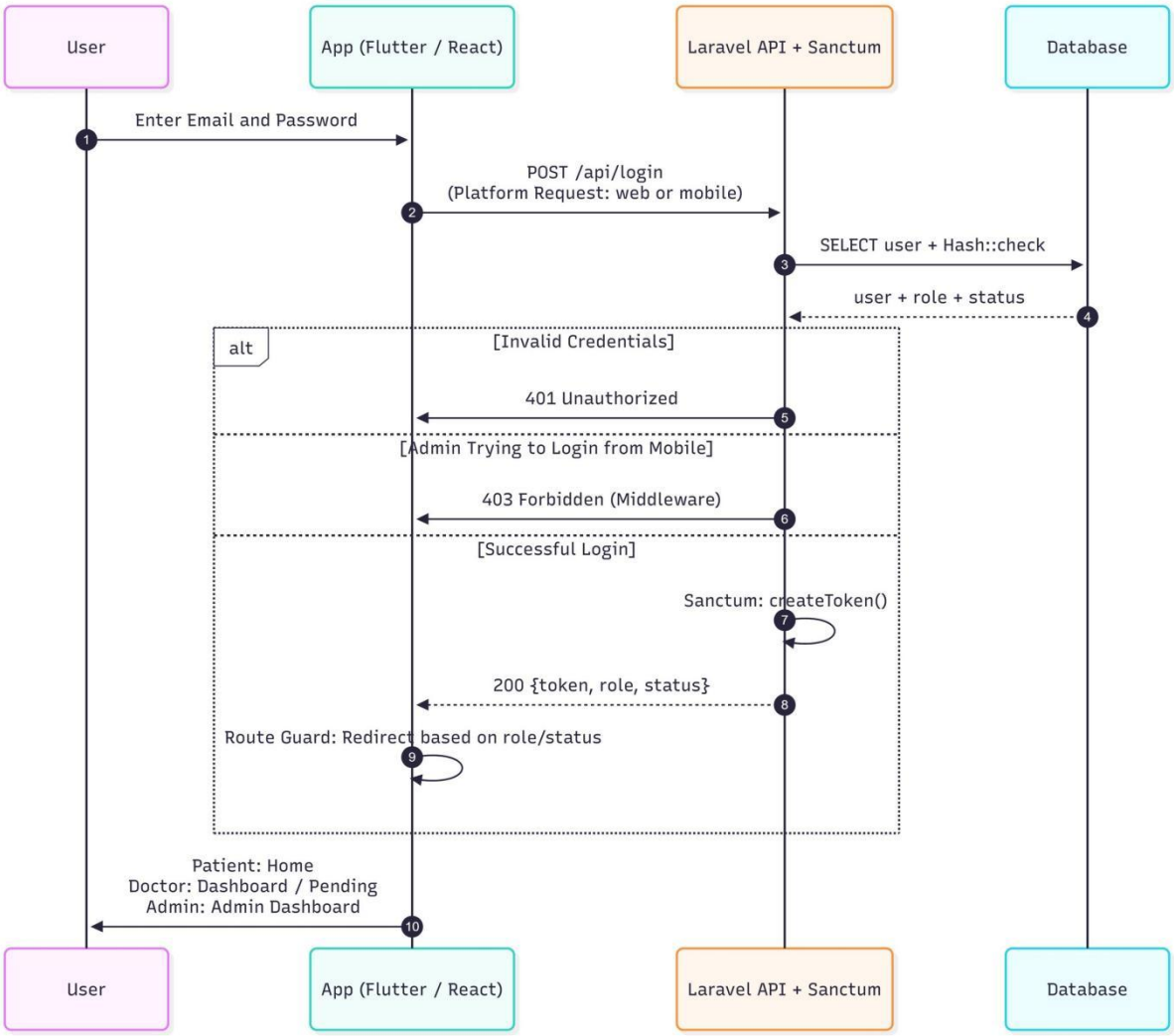
- **User Registeration Diagram**

Users (patient/doctor) submit their details, Laravel validates the input, hashes the password, stores it in the database, and generates a Sanctum token.



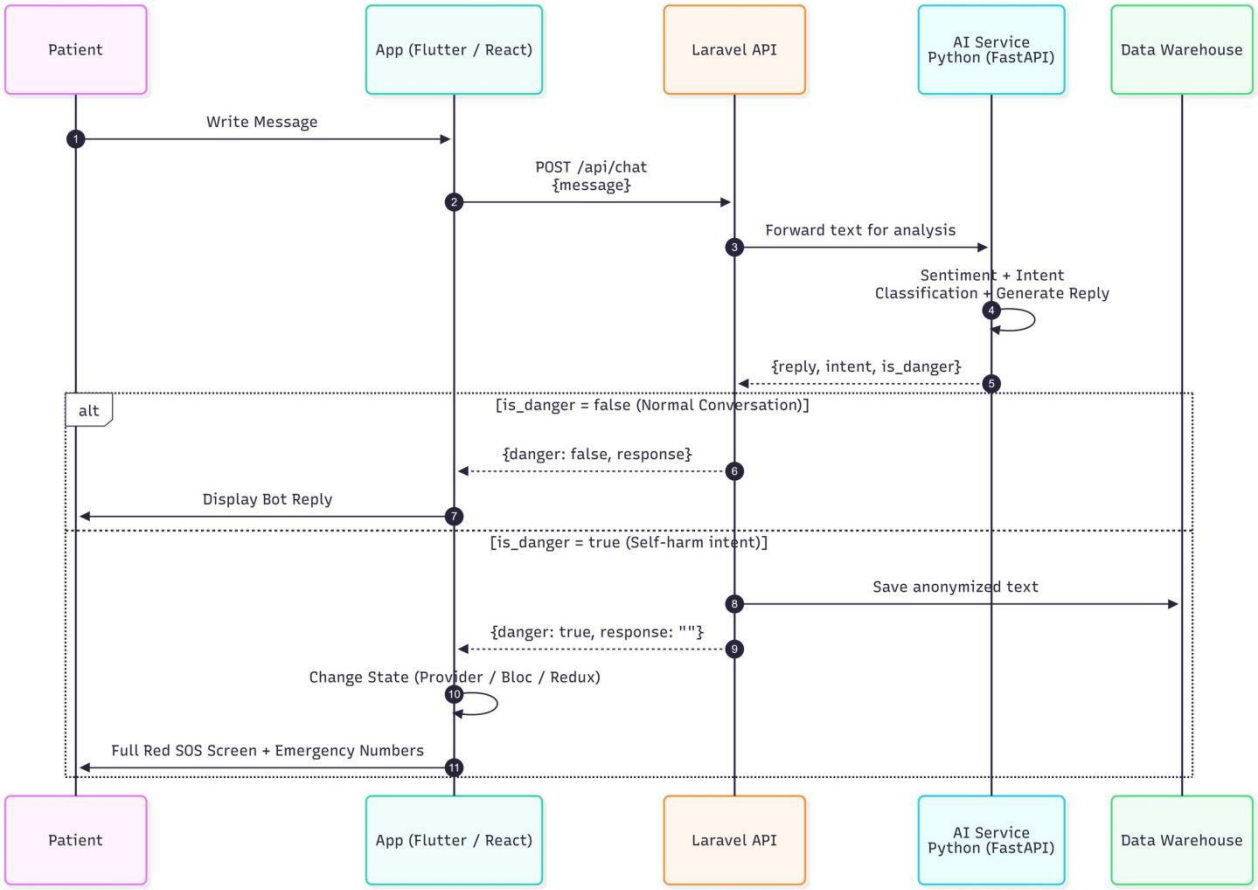
- **Login & Role Routing Diagram**

The system verifies credentials, blocks admins from the mobile app via middleware, and routes users to their correct screens based on their role.



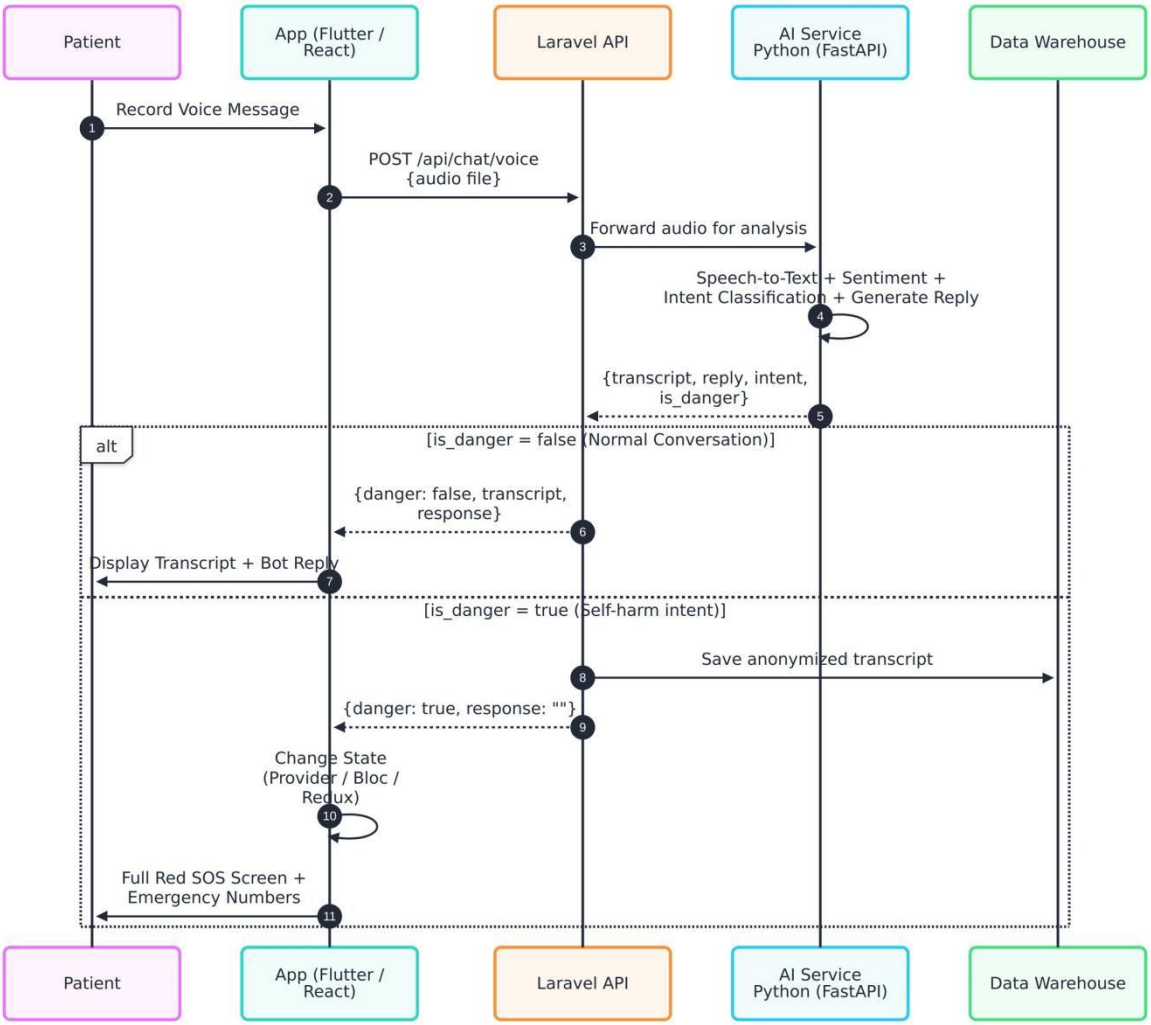
- **AI Chatbot & SOS Detect Diagram**

Messages are analyzed by the Python AI service; normal responses are displayed, while self-harm intent triggers an immediate red SOS screen.



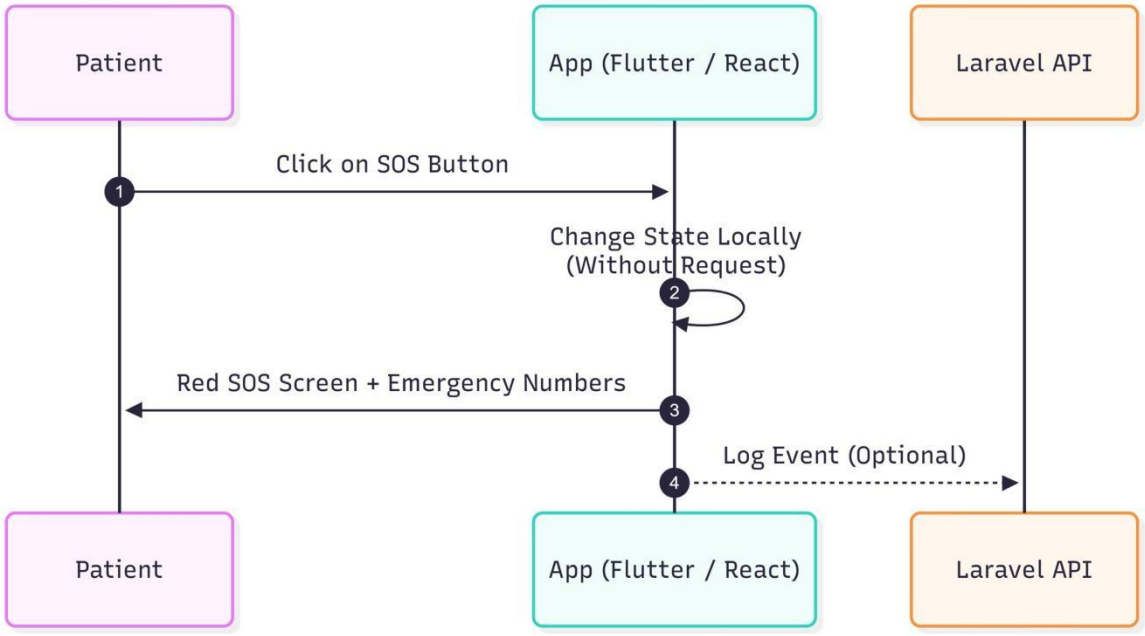
- **AI Voice Chatbot & SOS Detect Diagram**

The patient's voice message is sent to the Python AI service, transcribed to text, and analyzed in the same way as a typed message; normal responses are displayed together with the transcript, while self-harm intent triggers an immediate red SOS screen.



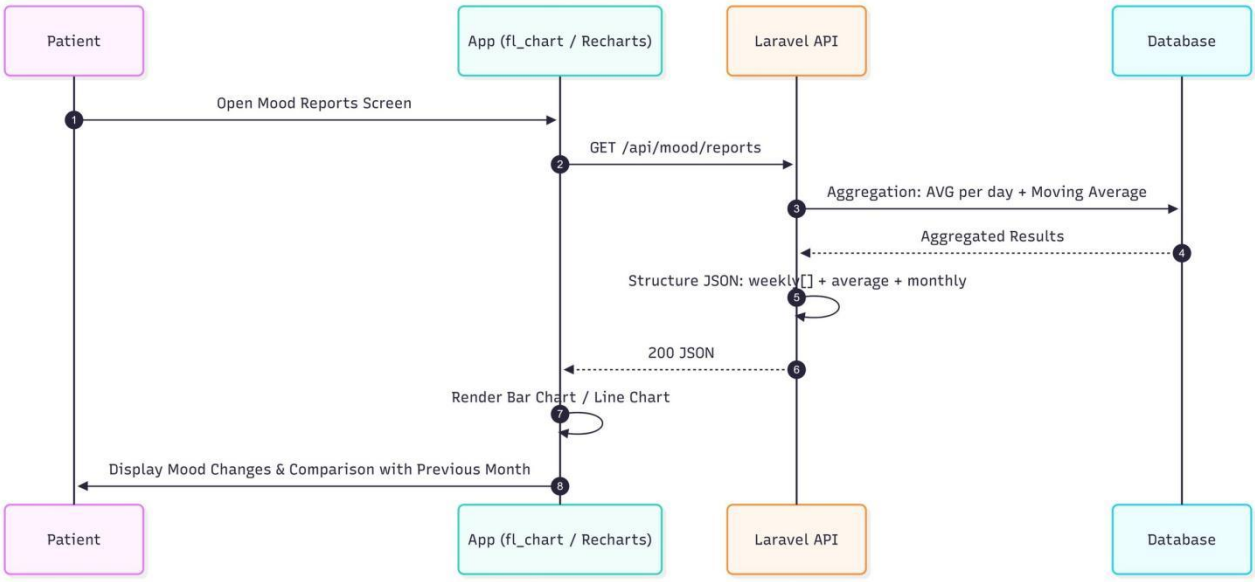
- **Direct SOS Button Diagram**

Tapping the emergency button locally triggers the full red SOS screen instantly without waiting for server requests.



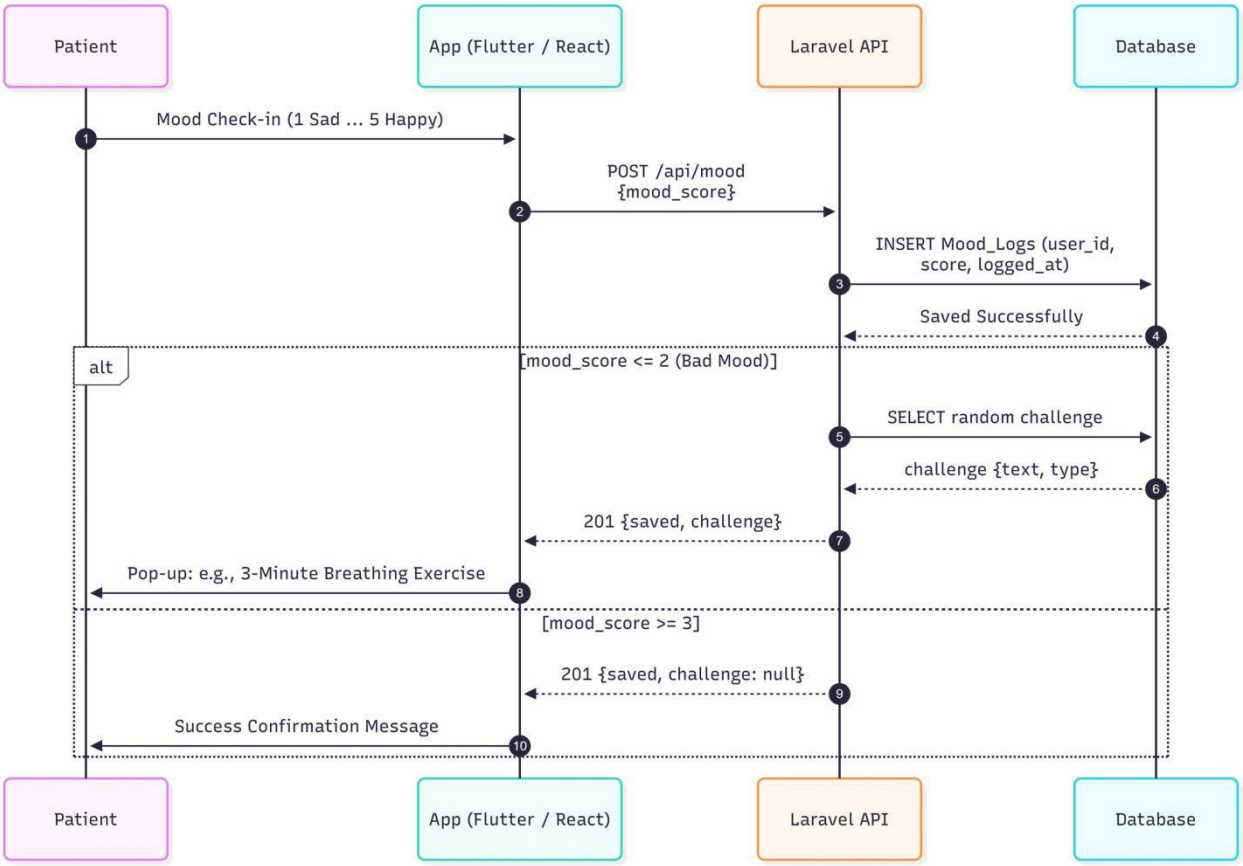
- **Mood Reports Diagram**

Laravel performs advanced SQL aggregations (averages and moving averages), returning structured JSON data for the app to render interactive charts.



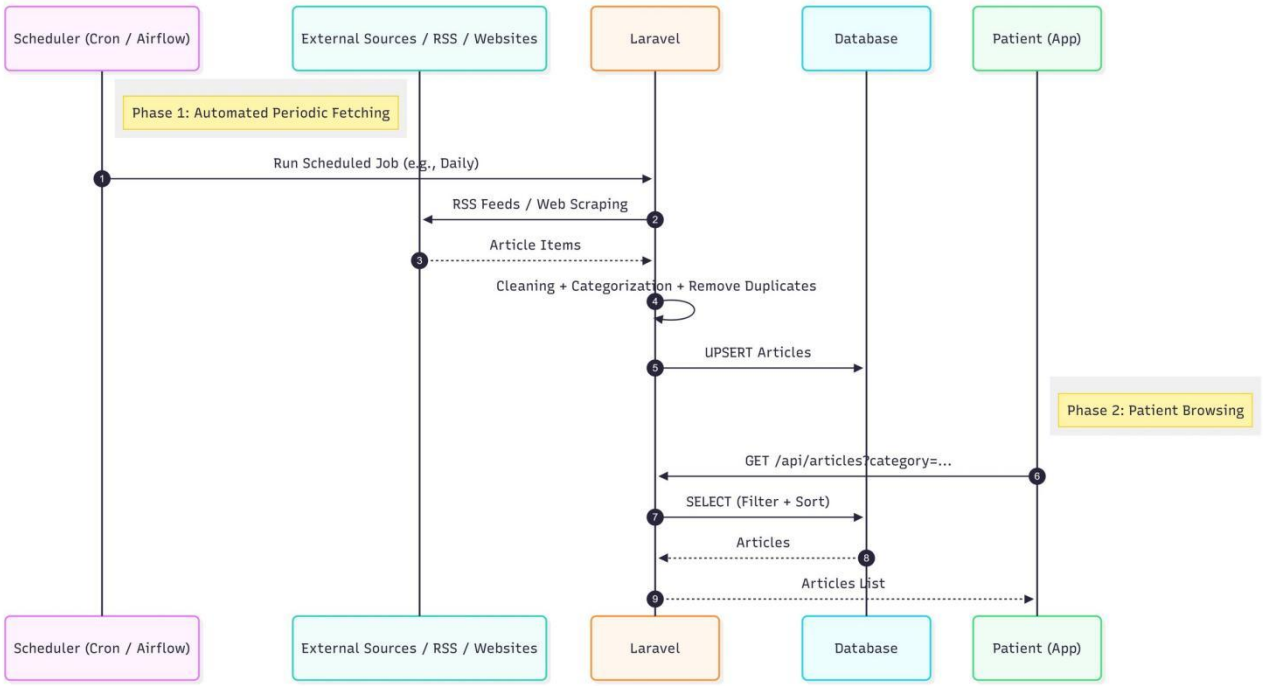
- **Mood Tracker & Pop-up Diagram**

Patients log their daily mood (1 to 5); if the mood score is low (<=2), the system fetches a random wellness challenge to display in a popup.



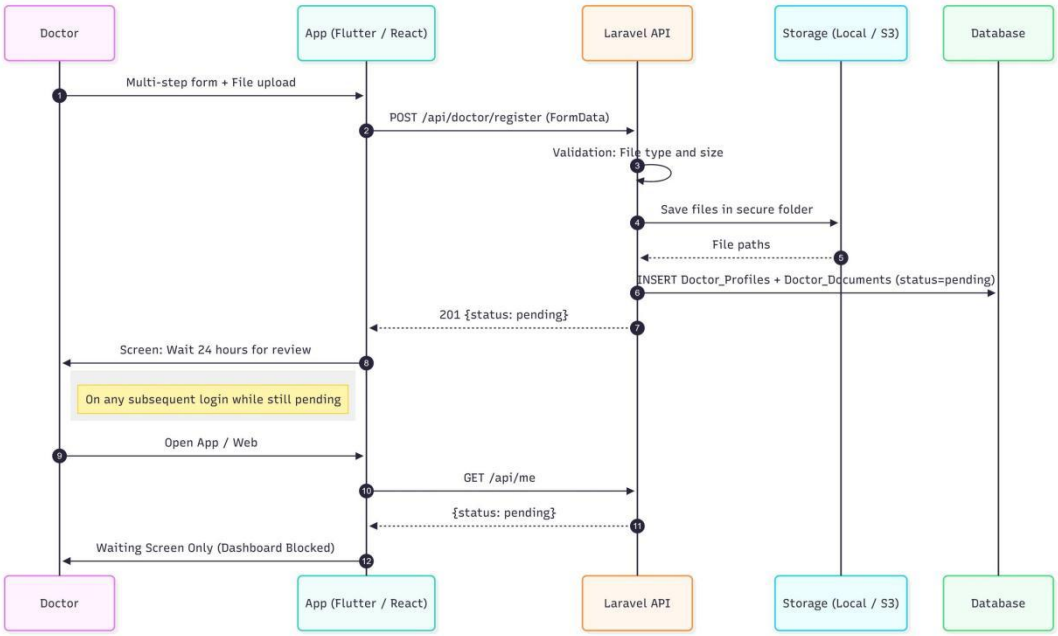
- **Articles Data Pipeline Diagram**

A cron job automatically fetches, cleans, and categorizes mental health articles, allowing patients to browse and filter them easily.

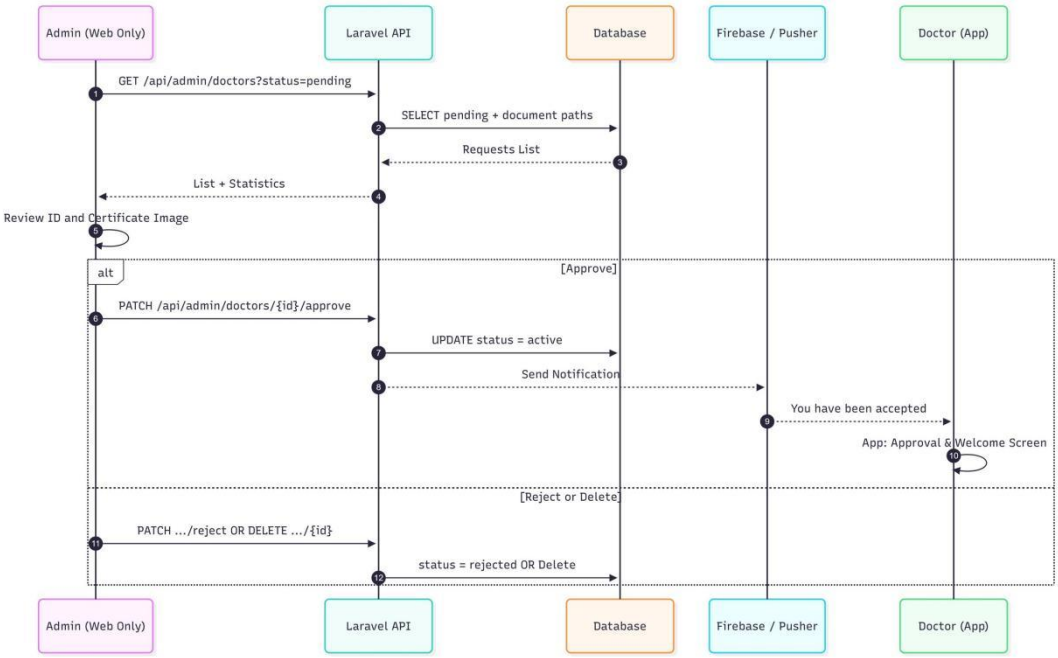


- **Doctor Onboarding & Verification Diagrams**

Doctors upload their ID and graduation certificate via FormData, storing them securely while setting their account status to pending.

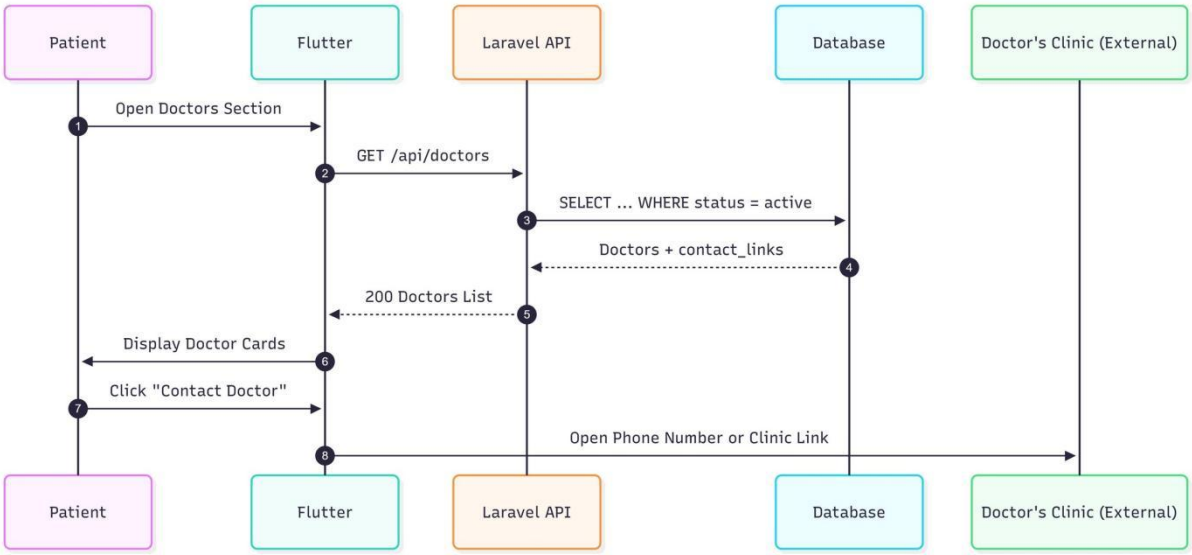


- **Doctor Verification Diagram**



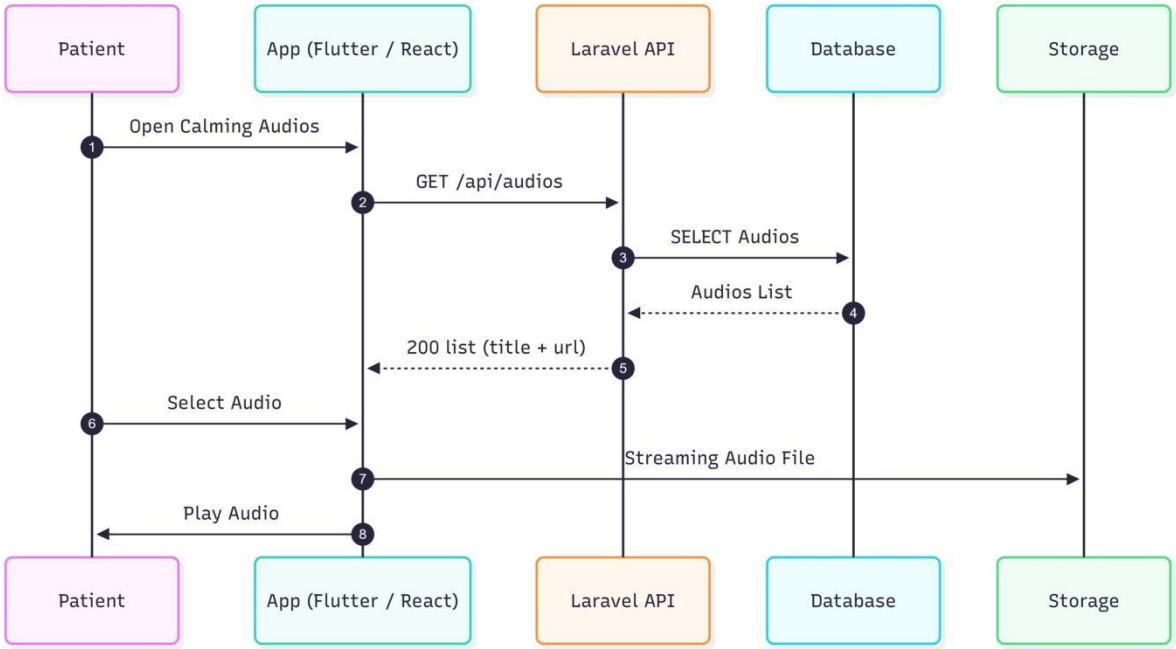
- **Patient-Side Doctor Browsing Diagram**

Patients query and view active doctors only, enabling direct external communication via phone numbers or clinic links.



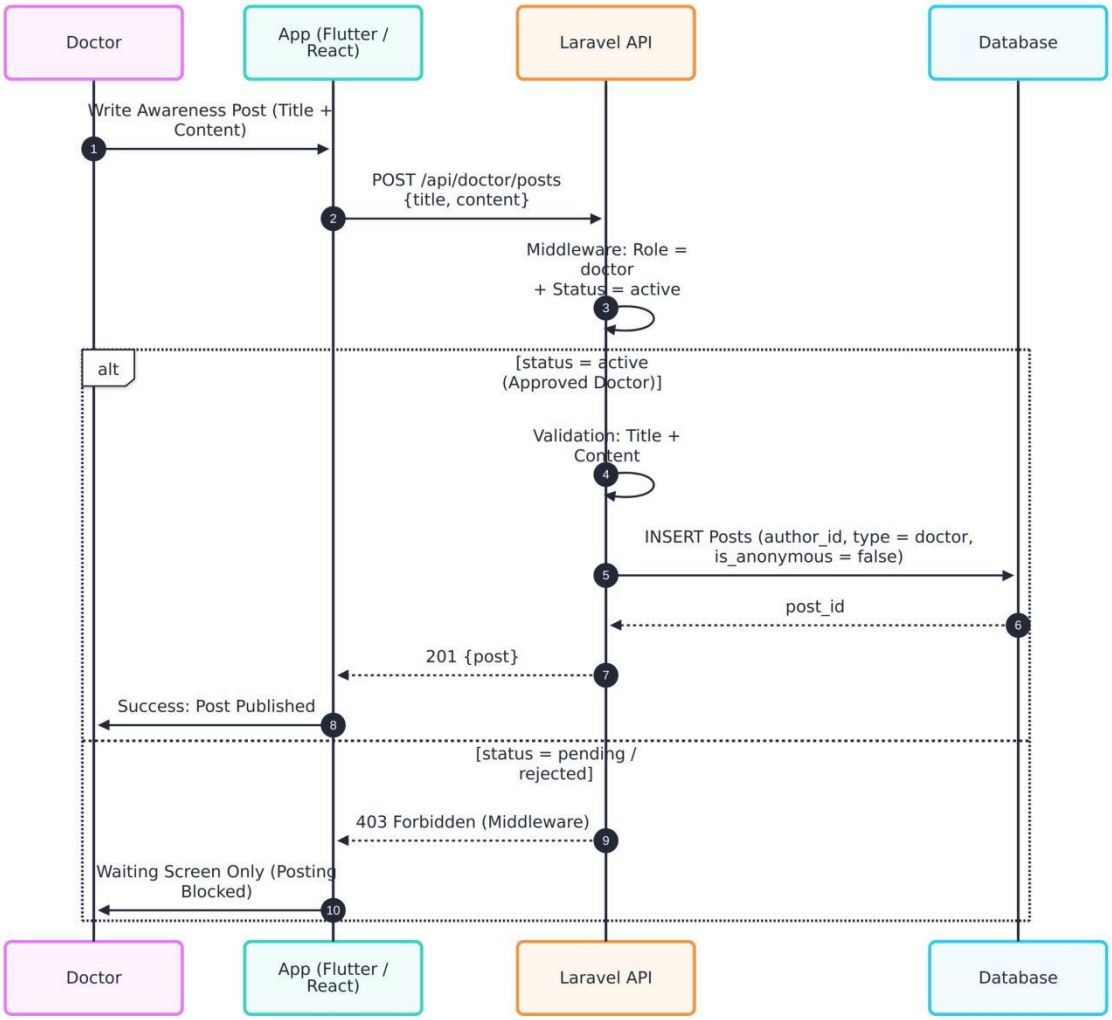
- **Audio Delivery Diagram**

The app fetches a static list of relaxing audio tracks from the database, allowing patients to stream and listen to them for relaxation.



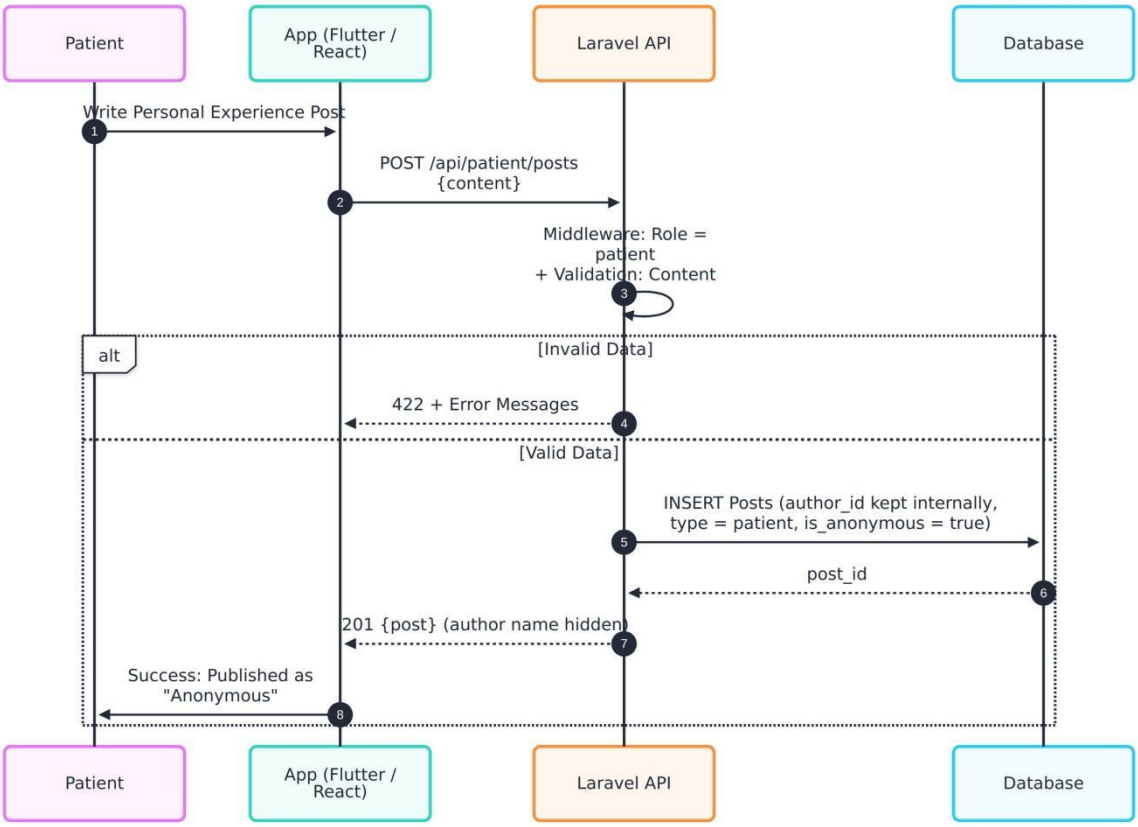
- **Doctor Awareness Post Diagram**

Approved doctors publish awareness posts and tips; the request is rejected by the middleware if the doctor's account is still pending or rejected.



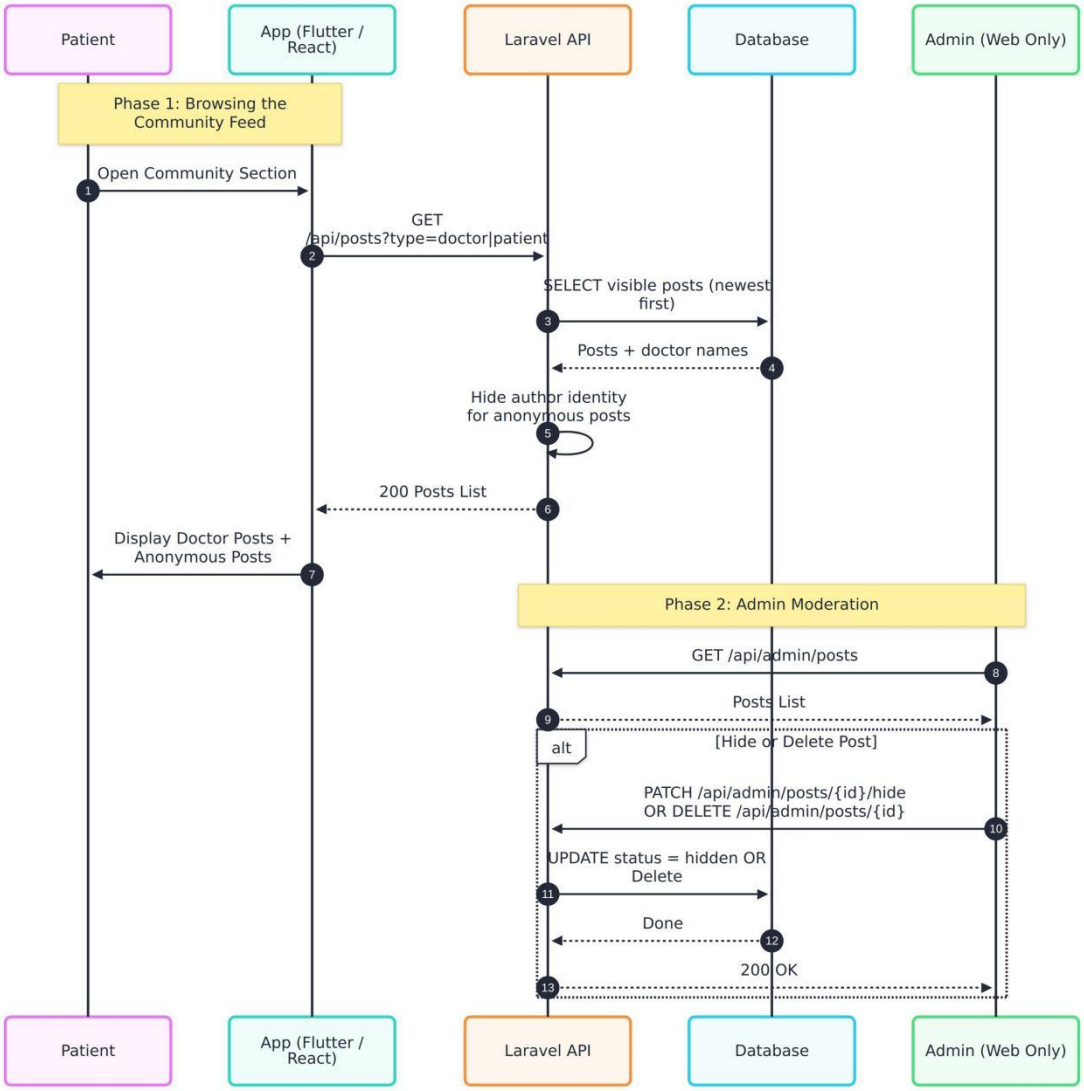
- **Anonymous Patient Post Diagram**

Patients share personal experiences to help others; the post is saved with an internal author reference for moderation, but it is published and returned without the author's name.



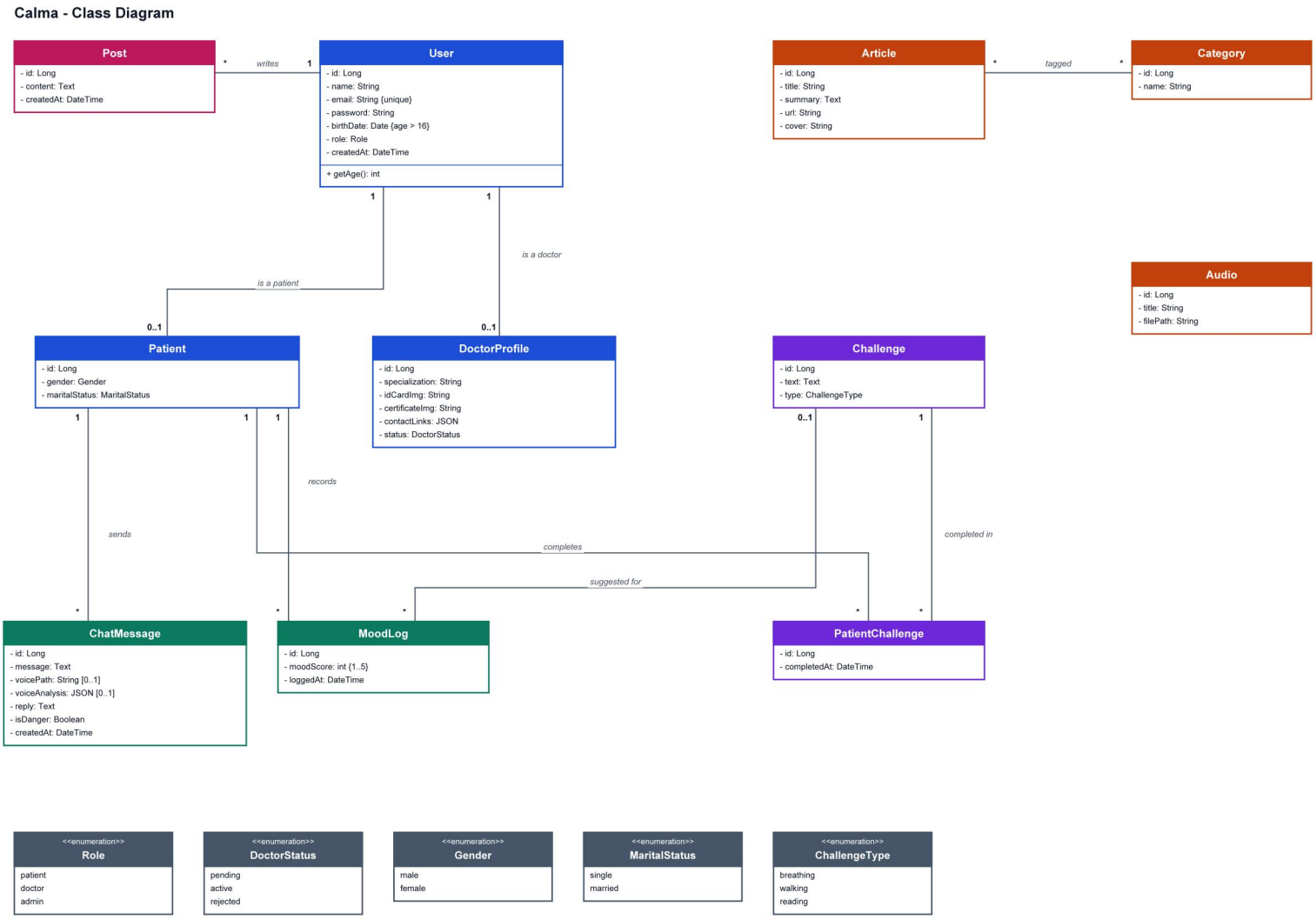
- **Community Feed & Moderation Diagram**

Patients browse doctor posts and anonymous posts, while the Admin can hide or delete any post from the web dashboard.



3.2.3 Class Diagram

This class diagram illustrates the object-oriented structure of the Calma system. It models the core entities—**User**, **Patient**, **Doctor**, and **Admin**—alongside functional components like **ChatBot**, **MoodLog**, **Challenge**, and **SOSScreen**, defining their attributes, methods, and relationships.



3.3 Conclusion

- **Requirements**

- Functional Requirements: The system supports three user roles (Patient, Doctor, Admin) managed via Laravel Sanctum authentication and strict role restrictions. It features an AI Chatbot (using a Python BERT microservice) that handles multi-lingual empathetic replies through text and voice messages (with Speech-to-Text transcription), self-harm detection (is_danger), and a secure red SOS emergency screen. It includes automated content scraping, secure document uploads, real-time notifications, mood tracking with pop-up wellness challenges, statistical reports, a doctor verification workflow, and a community awareness space where approved doctors publish posts and patients share experiences anonymously under Admin moderation.

- Non-Functional Requirements: Focuses on performance, token-based security, data privacy (including anonymous posting and voice recordings), content moderation, SOS reliability, calming usability/design, microservice scalability, multi-platform compatibility, Arabic/English localization, continuous availability, modular maintainability, and robust testability.

- **System Models:**

- DFD (Data Flow Diagram): Uses a context diagram and level one diagrams split into user management, doctor onboarding, patient services, and community posts to map out data flow.

- Sequence Diagrams: Model the runtime workflows for user registration, login route guards, AI chatbot safety detection (text and voice), direct SOS triggers, mood logging and reporting, article data pipelines, doctor onboarding/admin reviews, patient-side doctor browsing, audio streaming, doctor awareness posts, anonymous patient posts, and community feed moderation.

- Class Diagram: Illustrates the object-oriented architecture, mapping core entities (User, Patient, Doctor, Admin) and components (ChatBot, MoodLog, Challenge, SOSScreen, etc.) alongside their attributes, methods, and relationships.